

Automatic Periodic-Net Symmetry Discovery Specification

Stage 7R Revision: Reusable Barycentric Placement and Separate Ring Index

mdstats

2026-07-18

Contents

Purpose and boundary	1
Supported completeness domain	1
Theory	1
Public API	2
Construction algorithm	2
Ownership and persistence	3
Resource policy	3
LTA validation fixture	3
Explicit non-responsibilities	3
References	3

Purpose and boundary

`net_symmetry_discovery.py` discovers a complete generator set for one eligible `PeriodicNetView`, assembles the core `PeriodicNetSymmetry`, and optionally builds a separate `PrimitiveRingSymmetryIndex` when a compatible ring index is supplied.

Runtime/API target:

mdstats 0.19.19a0

The exact rational equilibrium placement is now owned by `periodic_barycentric.py`; discovery consumes and persists that source-bound result rather than owning a private solver.

Supported completeness domain

The first backend certifies completeness only when all conditions hold:

- full three-dimensional periodicity;
- one quotient component;
- translation rank three;
- translation-subgroup index one;
- collision-free exact barycentric coordinates modulo $\mathbb{Z}^3$; and
- at least one vertex star containing three linearly independent barycentric edge vectors.

Unsupported or resource-truncated inputs fail transactionally.

Theory

For quotient vertex i , exact barycentric coordinates $x_i \in \mathbb{Q}^3$ obey

$$x_i = \frac{1}{\deg(i)} \sum_{(j,t)} (x_j + \mathbf{t}),$$

with one anchor fixed to remove translational gauge. The reusable `PeriodicBarycentricPlacement` stores this exact solution and collision information.

Choose a deterministic source vertex and three independent incident vectors forming a matrix F . For every signature-compatible target frame F' , recover

$$A = F'F^{-1}.$$

Only integer unimodular matrices survive. Each surviving affine candidate is then reconstructed over all quotient vertices and validated against every explicit decorated multiedge.

Public API

```
@dataclass(frozen=True)
class NetSymmetryDiscoveryOptions:
    anchor_atom_index: int | None = None
    max_frame_trials: int = ...
    max_candidate_operations: int = ...
    max_group_operations: int = ...
    max_ring_composition_checks: int = ...
    max_barycentric_vertices: int = ...
    max_barycentric_fraction_bits: int = ...

@dataclass(frozen=True)
class BarycentricFrameIncidence:
    edge_position: int
    orientation: int

@dataclass(frozen=True)
class PeriodicNetSymmetryDiscovery:
    periodic_net_view_digest: str
    topology_graph_digest: str
    method: str
    anchor_atom_index: int
    source_frame: tuple[BarycentricFrameIncidence, ...]
    barycentric_placement: PeriodicBarycentricPlacement
    frame_trial_count: int
    candidate_operation_count: int
    generator_operation_indices: tuple[int, ...]
    symmetry: PeriodicNetSymmetry
    ring_symmetry: PrimitiveRingSymmetryIndex | None
    canonical_schema_version: str
    digest_algorithm: str
    digest: str

discover_periodic_net_symmetry(
    view: PeriodicNetView,
    *,
    ring_index: PrimitiveRingIndex | None = None,
    options: NetSymmetryDiscoveryOptions | None = None,
) -> PeriodicNetSymmetryDiscovery
```

Construction algorithm

1. Validate the view's periodic connectedness, rank, and subgroup index.
2. Build one exact PeriodicBarycentricPlacement under explicit rational-size resources.
3. Reject barycentric collisions for this backend.
4. Choose a deterministic source vertex/frame.
5. Enumerate signature-compatible target frames.
6. Recover candidate integer-unimodular lattice matrices exactly.
7. Reconstruct vertex images and explicit parallel-edge actions.
8. Validate each candidate as a view-bound automorphism.
9. Reduce accepted operations to a deterministic generator subset.
10. Build the core PeriodicNetSymmetry.
11. If a ring index is supplied, build a separate catalog-bound PrimitiveRingSymmetryIndex.

Ownership and persistence

The discovery result is orchestration/provenance. Its nested objects retain their own scientific identities:

```
PeriodicBarycentricPlacement  -- topology-derived rational placement
PeriodicNetSymmetry           -- core finite automorphism group
PrimitiveRingSymmetryIndex    -- optional ring-catalog-derived action
```

The ring index is optional and is not copied into the core group.

Schema:

```
mdstats.net-symmetry-discovery.v2
```

Deserialization validates the exact view and, when ring data are present, the supplied `PrimitiveRingIndex` and its catalog digest.

Resource policy

The options separately bound:

- exact barycentric system size;
- maximum numerator/denominator bit length;
- frame trials;
- accepted candidate operations;
- finite group order; and
- ring-action composition checks.

Exhaustion raises a resource exception and publishes no partial completeness claim.

LTA validation fixture

For the eligible unlabeled LTA T -net, the backend must recover 96 normalized representatives modulo translations. With the complete primitive-ring catalog through size eight, the derived ring index must cover all 82 rings and reproduce five ring orbits of sizes 6, 12, 16, 24, and 24.

Explicit non-responsibilities

The module does not compute:

- a Euclidean crystal metric or Cartesian embedding;
- distorted-frame space groups;
- noncrystallographic or collision-degenerate symmetry backends;
- natural tilings, faces, or cages; or
- ring strength.

References

1. O. Delgado-Friedrichs and M. O’Keeffe, *Identification of and Symmetry Computation for Crystal Nets*, Acta Cryst. A **59**, 351–360 (2003), DOI: 10.1107/S0108767303012017.
2. O. Delgado-Friedrichs, *Barycentric Drawings of Periodic Graphs*, in Graph Drawing, LNCS **2912**, 178–189 (2004), DOI: 10.1007/978-3-540-24595-7_17.